

**SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN**

Ime Prezime

**GENERIRANJE RAZINA IGRE
KORITENJEM GRAMATIKA GRAFOVA**

DIPLOMSKI RAD

Varaždin, 2024.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ź D I N

Ime Prezime

Matični broj: 35918/07–R

Studij: Informacijsko i programsko inženjerstvo

GENERIRANJE RAZINA IGRE KORITENJEM GRAMATIKA GRAFOVA

DIPLOMSKI RAD

Mentor:

doc. dr. sc. Titula Ime Prezime

Varaždin, rujan 2024.

Ime Prezime

Izjava o izvornosti

Izjavljujem da je ovaj diplomski rad izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autorica potvrdila prihvatanjem odredbi u sustavu FOI Radovi

Sažetak

Ovaj rad opisuje implementaciju sustava za proceduralno generiranje razina u računalnim igrama koristeći gramatike grafova (engl. Graph Grammars). Sustav je implementiran u programskom jeziku Prolog s vizualizacijom u Pythonu (PyGame). Rad proučava teorijsku podlogu gramatika grafova i njihovu primjenu u stvaranju kompleksnih i varijabilnih struktura tamnica (dungeons).

Ključne riječi: gramatike grafova; proceduralno generiranje sadržaja; Prolog; Python; razvoj igara; roguelike

Sadržaj

1. Uvod	1
2. Teorijska podloga	2
2.1. Proceduralno generiranje sadržaja	2
2.2. Gramatike grafova	2
2.3. Logičko programiranje i Prolog	2
3. Opis implementacije	3
3.1. Arhitektura sustava	3
3.2. Generator u Prologu	3
3.2.1. Definicija gramatike	3
3.2.2. Algoritam ekspanzije	3
3.3. Integracija i Vizualizacija	4
4. Kritički osvrt	5
4.1. Prednosti	5
4.2. Nedostaci	5
5. Prikaz rada aplikacije	6
6. Zaključak	7
Popis literature	8
Popis slika	9
Popis isječaka koda	10

1. Uvod

Računalne igre su kompleksni softverski sustavi koji zahtijevaju velike količine sadržaja kako bi održali interes igrača. Jedan od ključnih elemenata mnogih igara, posebno onih u žanru *roguelike*, je dizajn razina. Ručno dizaniranje razina je vremenski zahtjevan proces koji ograničava količinu sadržaja koju razvojni tim može proizvesti. Kako bi se riješio ovaj problem, koristi se proceduralno generiranje sadržaja (engl. *Procedural Content Generation - PCG*).

Ovaj rad bavi se proceduralnim generiranjem razina u igri korištenjem gramatika grafova (engl. *Graph Grammars*). Motivacija za ovaj pristup leži u mogućnosti definiranja strukture razine putem skupa logičkih pravila, što omogućuje generiranje topološki ispravnih i zanimljivih tlocrta tamnica. Za razliku od jednostavnih nasumičnih algoritama, gramatike grafova nude bolju kontrolu nad strukturom i povezanošću prostorija [1].

Cilj rada je implementirati sustav koji koristi deklarativnu paradigmu (Prolog) za generiranje strukture razine na temelju gramatike, te imperativnu/objektno-orijentiranu paradigmu (Python) za vizualizaciju i interakciju s igračem.

2. Teorijska podloga

2.1. Proceduralno generiranje sadržaja

Proceduralno generiranje sadržaja (PCG) odnosi se na stvaranje sadržaja igre automatski, putem algoritama, s ograničenim ili neizravnim unosom korisnika [2]. PCG se koristi za generiranje različitih vrsta sadržaja, uključujući texture, modele, glazbu, priču i razine. U kontekstu ovog rada, fokus je na generiranju razina, odnosno tlocrta prostora (tamnica).

2.2. Gramatike grafova

Gramatike grafova su formalizam za specificiranje klasa grafova i operacija nad njima. Slično kao što Chomskyjeve gramatike definiraju pravila za generiranje nizova znakova (simbola), gramatike grafova definiraju pravila za generiranje grafova [1].

Gramatika grafa sastoji se od početnog grafa (aksioma) i skupa produkcijskih pravila (pravila prepisivanja). Svako pravilo definira podgraf koji se traži u trenutnom grafu (lijeva strana pravila) i podgraf kojim se on zamjenjuje (desna strana pravila).

Primjena gramatika grafova u generiranju razina omogućuje dizajnerima da definiraju "recente" za strukturu razine. Na primjer, pravilo može reći: "Ako postoji soba s blagom, dodaj sobu s čudovištem ispred nje". Ovo osigurava da generirane razine imaju smisla u kontekstu igrivosti (engl. *gameplay*).

2.3. Logičko programiranje i Prolog

Prolog (Programming in Logic) je deklarativni programski jezik koji se temelji na predikatnoj logici prvog reda. Program u Prologu sastoji se od činjenica i pravila. Prolog je izuzetno pogodan za rješavanje problema koji uključuju simboličku manipulaciju, pretraživanje prostora stanja i zaključivanje, što ga čini idealnim alatom za implementaciju gramatika grafova.

U ovom radu Prolog se koristi za definiranje pravila gramatike i izvođenje procesa generiranja pretraživanjem i zamjenom simbola (čvorova grafa).

3. Opis implementacije

Sustav se sastoji od dviju glavnih komponenti: generatora razina implementiranog u Prologu i grafičkog sučelja te logike igre implementiranih u Pythonu. Ove dvije komponente komuniciraju putem biblioteke *pyswip* (ili subprocess poziva), gdje Python šalje zahtjev za generiranjem, a Prolog vraća strukturu razine.

3.1. Arhitektura sustava

Arhitektura je podijeljena na:

- **Backend (Prolog):** Odgovoran za logičko generiranje tlocrta tamnice, određivanje tipova soba i njihovih veza.
- **Frontend (Python/PyGame):** Odgovoran za vizualizaciju, upravljanje stanjem igre (kretanje igrača, borba) i interakciju s korisnikom.

3.2. Generator u Prologu

Jezgra generatora nalazi se u datoteci `dungeon_generator.pl`. Generiranje započinje definiranjem gramatike grafa.

3.2.1. Definicija gramatike

Gramatika je definirana nizom pravila prepisivanja. Primjer pravila iz koda: Ova pravila određuju koje nove sobe (čvorove) možemo stvoriti iz postojeće sobe određenog tipa. Na primjer, iz 'start' sobe možemo granati u 'combat' ili 'event' sobu.

3.2.2. Algoritam ekspanzije

Proces generiranja koristi predikat `expand_dungeon/4`. Algoritam funkcionira na sljedeći način:

1. Zapочиње s početnom sobom (tip *start*) u redu čekanja.

```
1 grammar_rule(start,    [combat, event]).
2 grammar_rule(combat,   [combat, treasure]).
3 grammar_rule(event,    [treasure, empty]).
4 grammar_rule(empty,    [combat]).
5 grammar_rule(treasure, [boss]).
6 grammar_rule(boss,     []).
```

Isječak koda 1: Pravila gramatike za generiranje soba

```
1 # Primjer pseudo-koda za dohvat
2 rooms = prolog.query("generate_dungeon(Seed, 20)")
3 for room in rooms:
4     draw_room(room)
```

Isječak koda 2: Integracija Pythona i Prologa

2. Uzima sobu iz reda, provjerava njen tip i dohvaća primjenjiva pravila gramatike.
3. Za svaki novi tip sobe iz pravila, pokušava pronaći slobodan susjedni prostor na rešetki (grid) pozivom `spawn_neighbors/9`.
4. Ako je prostor slobodan, nova soba se stvara, povezuje s roditeljem i dodaje u red za daljnju ekspanziju.
5. Proces se ponavlja dok se ne dosegne željeni broj soba.

3.3. Integracija i Vizualizacija

Python skripta pokreće Prolog generator i dohvaća rezultate u obliku liste soba i veza. Svaka soba se vizualizira kao pločica na ekranu, a hodnici (veze) se iscrtavaju između njih. Dodatno, sadržaj soba (neprijatelji, predmeti) se također generira u Prologu (`generate_room_content/1`) i prenosi u Python igru.

4. Kritički osvrt

Korištenje gramatika grafova pokazalo se kao robustan pristup za generiranje tamnica.

4.1. Prednosti

Glavna prednost je odvajanje strukture (gramatike) od algoritma generiranja. Promjenom samo nekoliko linija koda u `grammar_rule` predikatima, možemo drastično promijeniti "osjećaj" i topologiju tamnice, npr. stvarati linearne hodnike ili razgranate labirinte. Također, Prologova *backtracking* priroda olakšava implementaciju ograničenja.

4.2. Nedostaci

Glavni nedostatak je potencijalna eksponencijalna složenost ako se pravila ne ograniče. Također, komunikacija između Prologa i Pythona može uvesti određeno kašnjenje (overhead), iako za generiranje razina to nije kritično jer se događa samo jednom pri učitavanju.

5. Prikaz rada aplikacije

U nastavku su prikazani primjeri generiranih tamnica.

[OVDJE UBACITI SLIKU GENERIRANE TAMNICE]

Slika 1: Primjer generirane tamnice s 20 soba

Aplikacija omogućuje igraču kretanje kroz generirane sobe, borbu s neprijateljima i skupljanje predmeta.

6. Zaključak

U ovom radu uspješno je implementiran sustav za proceduralno generiranje razina koristeći gramatike grafova u Prologu. Sustav demonstrira moć deklarativnog programiranja u rješavanju problema generiranja strukture. Iako postoje izazovi u integraciji s imperativnim jezicima, konačni rezultat je fleksibilan i proširiv generator koji može poslužiti kao temelj za složenije igre.

Popis literature

- [1] J. Dormans, „Adventures in level design: generating missions and spaces for action adventure games,” *Proceedings of the 2010 Workshop on Procedural Content Generation in Games*, 2010., str. 1–8.
- [2] J. Togelius, G. N. Yannakakis, K. O. Stanley i C. Browne, „Search-based procedural content generation: A taxonomy and survey,” *IEEE Transactions on Computational Intelligence and AI in Games*, sv. 3, br. 3, str. 172–186, 2011.

Popis slika

1.	Primjer generirane tamnice s 20 soba	6
----	--	---

Popis isječaka koda

1.	Pravila gramatike za generiranje soba	3
2.	Integracija Pythona i Prologa	4